Table of Contents

Open-Source DOCX Editors, Re-verified Against Code and Measured by Round-Trip (2026-10-29)
- TL;DR
- Method and Its Limits
  - What was done
  - Evidence tags
  - Pinned commits (all main/master as of 2026-10-29)
  - How the twelve were chosen
  - Limits of this trial
- The Corpus: fixture/complex.docx
- Harnesses: the exact code path each project was driven through
- Measured Results
  - Round-trip scorecard
  - What each project actually lost
  - Second pass (idempotence)
- Word Compatibility of the Corpus
- Executive Summary: Rankings
- Master Comparison Table
- Preserved-vs-Dropped Matrix (measured)
- Per-Repository Findings
  1. SuperDoc V2 — superdoc/docx-editor
  2. stella/folio
  3. genspark-ai/genoffice
  4. eigenpal/docx-editor
  5. DemoMacro/office-open
  6. DemoMacro/docen
  7. storytold/wordcraft
  8. Hufe921/canvas-editor + canvas-editor-plugin
  9. Fidus Writer
  10. docs-plus/docs.plus
  11. udecode/plate — Plate.js (added 2026-10-30)
- Changes from the Previous Report
- Recommendations
- Caveats
- Reproducing This

Open-Source DOCX Editors, Re-verified Against Code and Measured by Round-Trip (2026-10-29)

This edition does what the previous one could not: it **downloads every editor's code, runs each editor's own import→export path over a purpose-built adversarial document, and scores the result**. Every cell that previously read NOT VERIFIED IN CODE is now either a source citation or a measurement.

Headline result: only three projects survive the corpus intact — **SuperDoc (100%)**, **Folio (98.5%)** and **eigenpal (98.5%)**. **GenOffice** matches Folio. **office-open** and **Docen** regenerate the package and silently drop revision metadata. **WordCraft**keeps the chart, SmartArt and OLE payload too — as the six editors above also do — but loses the revision model. **Fidus Writer**, **canvas-editor ✦ plugin** and **docs.plus** are not round-trip tools in any meaningful sense.

A correction to the previous report's headline: it ranked **eigenpal first on OOXML fidelity**. Measured, SuperDoc is first and eigenpal is third — eigenpal's canonical tree flattens mc:AlternateContent to its mc:Choice branch and drops the mc:Fallback content.

> **Second edition (2026-10-30): a twelfth editor, and a search for the rest.**
>
> **Plate.js** (udecode/plate, **16,680★** — the second-most-starred project in this trial) has been added: cloned at 08b1571f2, driven through its own headless importDocx → exportDocx path with **45 base plugins** and Plate's own **static presentation kit**, and scored on the same 97 checks. It scores **20.5%** — last in the trial — but it is the **only editor that refuses a malformed document instead of corrupting it**, and the only one that reports *what* it dropped rather than dropping it silently. Those two findings are covered in full in the companion reports.
>
> The selection was widened at the same time: a GitHub API sweep over the docx-editor, word-processor and ooxml topics plus a star-count survey of every candidate produced the shortlist in §"How the twelve were chosen". **Star count turned out to have no relationship to fidelity** — the two most-starred projects in the trial (WordCraft, 16,740★, and Plate, 16,680★) score 83.0% and 20.5%, while the winner has 1,117.

TL;DR

- **Best measured fidelity: SuperDoc V2.** 100% on the 97-point corpus, 31/31 critical features, 100% text retention, all 35 package parts preserved, and a genuine re-save (8 parts re-serialized, 27 untouched parts passed through byte-identically). It is the only project with **zero** failed checks and the only one whose second pass is also a perfect, byte-identical fixed point.
- **Four of the eight re-tested editors reach a byte-identical fixed point** — SuperDoc, Folio, eigenpal and GenOffice all reproduce their own output exactly when it is fed back through their code path. office-open and Docen do not: **their own output is malformed XML**, because the chart and SmartArt regeneration paths drop the local a: namespace binding their first pass emitted.
- **Folio and eigenpal are effectively tied on features (98.5%), but Folio keeps 100% of the text and eigenpal keeps 97.6%** — because eigenpal flattens mc:AlternateContent to its mc:Choice branch and drops the fallback text. Folio's only loss is w14:paraId/w14:textId.
- **GenOffice also scores 98.5%** and preserves all 35 parts, but regenerates one block of the fixture (the table), which silently discards the text inside vertically-merged continuation cells.
- **office-open (95.5%) and Docen (92.0%) are full-regeneration pipelines.** They rebuild the package from a parsed model, so unknown markup (w16du:dateUtc, w14:paraId/textId, the w16du namespace) is dropped by construction. **Both are also not idempotent: re-importing their own output produces word/document.xml with an unbound a: prefix — malformed XML.**
- **WordCraft (83.0%) is the most interesting failure.** It keeps the chart, the SmartArt diagram, the OLE object, the embedded workbook and the image — **as SuperDoc, Folio, GenOffice, eigenpal, office-open and Docen also do** — yet it drops w:proofErr, w:fldSimple, w:altChunk, mc:AlternateContent, w16du:dateUtc, the w14 paragraph identifiers and row-level revisions. It is the only payload keeper that loses the revision model, and the only one that drops payload *parts* of its own (word/charts/style1.xml, word/charts/colors1.xml). Its writer is a regeneration; its payload passthrough is what saves it.
- **Plate.js (20.5%) is the most instructive result, in both directions.** Its DOCX engine is a genuine, carefully built piece of work: 45 plugins, a headless import/export pair, a typed diagnostic vocabulary, and a **closed 'passive vocabulary' that governs whether an unchanged document may be exported byte-exactly**. Measured against the corpus it keeps **no** parts byte-identically, drops 24 of 35, renders 2 pages instead of 6, and scores last. But **it is the only editor in the trial that reports what it lost**: 38 typed diagnostics naming each dropped construct, and a default lossPolicy of 'reject' that makes it **refuse** a document it cannot represent losslessly instead of writing a degraded one.
- **LibreOffice, as a calibration baseline, scores 86.0% / 77.4% critical.** Any open-source editor that cannot beat LibreOffice on a Word-authored document is not a fidelity tool.

- **The three weakest are not round-trip pipelines.** Fidus Writer (23.0%) has a DOCX importer that **drops all tables**; canvas-editor + plugin (24.0%) is a canvas editor with a DOCX export format and **crashes on `w:vMerge`**; docs.plus (21.0%) is an HTML bridge whose importer is `mammoth`, which flattens tracked changes to their accepted state — insertion text survives as ordinary text, deletion and move-out text is dropped, and every piece of revision markup is destroyed.
- **Popularity does not predict fidelity.** Star counts for the twelve projects are in §"How the twelve were chosen". The two most-starred (WordCraft 16,740★, Plate 16,680★) score 83.0% and 20.5%; the winner has 1,117★; the project that ties for second place has 31★.

Method and Its Limits

What was done

1. **Every repository was cloned** (shallow, at pinned commits) and read. Fifteen checkouts, 700 MB of source, covering all nine editors plus four plugin/tooling repos.
2. **A purpose-built adversarial fixture was authored as raw OOXML**: `fixture/complex.docx`, 35 package parts, exercising 31 critical exception cases (§ *The Corpus* below).
3. **Each editor's own import→export path was executed headlessly** — not a re-implementation, the project's own entry point (§ *Harnesses* below for the literal commands).
4. **A 97-check scorer** diffed input against output at package-part level, element level, attribute level and text level.
5. **Every output was re-opened with LibreOffice** to confirm the result is still a valid, openable document.
6. **The corpus was opened in Microsoft Word 16.115**, and the defects Word refuses were found by bisecting `word/document.xml` against Word itself (§ *Word Compatibility* below). The corpus now opens in Word, and so does every file this edition delivers.

Evidence tags

[CODE] read in source at a pinned commit · [TEST] read in a test file · [DOCS] README/docs · [MEASURED] produced by this trial · [RELEASE/ISSUE] release notes, issues, PRs · [THIRD-PARTY] blogs and mirrors.

Pinned commits (all main/master as of 2026-10-29)

Checkout	Commit	Date
udecode/plate	08b1571f2	2026-10-30 (branch next)
eigenpal/docx-editor	c6f8f0fdb	2026-10-29
stella/folio	c83cfc4b0	2026-10-29
genspark-ai/genoffice	a538d0429	2026-10-29
storytold/wordcraft	aadb077e0	2026-10-28
superdoc/docx-editor	f52c959b2	2026-10-29
Hufe921/canvas-editor	8b9 477 218	2026-10-28
Hufe921/canvas-editor-plugin	0e8a1b36c	2026-10-10
DemoMacro/docen	1890b15aa	2026-10-29
DemoMacro/office-open	567addfd4	2026-10-30
akademi-ink/office-open	cc137a0eb	2026-10-28
docs-plus/docs.plus	a9b0c69e8	2026-10-24
fiduswriter/fiduswriter	e14cc456a	2026-10-21
fiduswriter/fiduswriter-document-ts	dc0981db4	2026-10-21

Checkout	Commit	Date
fiduswriter/fiduswriter-cli-ts	da64f9340	2026-10-21
fiduswriter/fiduswriter-pandoc-plugin-ts	b0ca59 400	2026-10-21

How the twelve were chosen

The first eleven were chosen by reading the DOCX-editor landscape and keeping what could be driven headlessly. The twelfth — **Plate.js** — was added after a systematic sweep, and the sweep is worth recording because of what it found.

The search used the GitHub API over three queries and three topic listings (docx-editor, word-processor, ooxml), sorted by stars, plus a direct star-count query for every candidate that survived. The result set, with the star counts **measured on 2026-10-30** rather than quoted from a search snippet:

Project	★	Language	Licence	In this trial?
storytold/wordcraft	**16,740**	Rust	Apache-2.0	yes — #8
udecode/plate	**16,680**	TypeScript	NOASSERTION	**yes — #12, added 2026-10-30**
genspark-ai/genoffice	**11,384**	TypeScript	Apache-2.0	yes — #4
dolanmiu/docx	5,984	TypeScript	MIT	no — a **generator**, not an editor: it writes .docx from a JS model and cannot read one
ONLYOFFICE/DocumentServer	7,066	Shell/C++	AGPL-3.0	no — a full office server, not a library; its round trip is a document-server pipeline, not an embeddable editor API
Hufe921/canvas-editor	**5,246**	TypeScript	MIT	yes — #9
superdoc/docx-editor	**1,117**	TypeScript	AGPL-3.0	yes — #1
fiduswriter/fiduswriter	**569**	TypeScript	AGPL-3.0	yes — #10
stffnb/edentext	562	TypeScript	—	no — a browser word processor; no library round-trip API
eigenpal/docx-editor	**536**	TypeScript	Apache-2.0	yes — #5
openooxml/betteroffice	328	Rust	—	no — not yet runnable headlessly against the corpus
Hufe921/canvas-editor-plugin	**131**	TypeScript	MIT	yes — #9 (with canvas-editor)
JSv4/Docxodus	106	C#	MIT	no — .NET/WASM; a different toolchain from the rest of the trial
docs-plus/docs.plus	**95**	TypeScript	MIT	yes — #11
CasualOffice/docs	68	TypeScript	—	no — self-hosted server with a co-editing stack; no headless round-trip API
DemoMacro/docen	**67**	TypeScript	MIT	yes — #7
ant981 228/cardmirror	45	TypeScript	—	no — a policy-debate tool, not a general DOCX editor
DemoMacro/office-open	**39**	TypeScript	MIT	yes — #6
CasualOffice/opendoc	34	Rust	Apache-2.0	no — a new, small engine

Project	★	Language	Licence	In this trial?
stella/folio	**31**	TypeScript	Apache-2.0	yes — #3

The star counts do not predict anything about fidelity. The winner, SuperDoc, has **1,117★**. The project that ties for second place, Folio, has **31★** — fewer than a hundred people have starred it, and it loses a single check. The two most-starred projects in the trial, WordCraft and Plate, score **83.0%** and **20.5%**. Meanwhile the projects with the most stars of all are mostly *not* editors: ONLYOFFICE/DocumentServer (7,066★) is a server and dolanmiu/docx (5,984★) is a generator.

Two structural candidates were **excluded on grounds, not on stars**, and the exclusions are recorded so the trial's boundary is auditable: a *generator* (dolanmiu/docx, docxtemplater, phpword, python-docx, unioffice) writes documents but cannot read one, so there is nothing to round-trip; and a *server* (ONLYOFFICE/DocumentServer, CasualOffice/docs) renders and edits over a document-server protocol, so its "round trip" is a different product from a library's. Everything that could be driven headlessly through a real import→export path was in scope.

Limits of this trial

- **One document.** The corpus is deliberately hostile, but it is one document; it is not a corpus survey.
- **Import→export with no edits.** Every project was exercised on its *round trip*, not on editing. Selective-save optimisations that only trigger on a dirty block are therefore not distinguished from unconditional regeneration — except where a project documents otherwise.
- **Word is used as a validator, not as an oracle.** The corpus is checked against the ECMA-376 schemas and opened in Microsoft Word 16.115, but the *outputs* were judged by the scorer and by LibreOffice, not by Word. A document that Word opens but that the scorer flags is still a loss.
- **Star counts and issue counts were not re-measured**; they change daily and are not the subject of this edition. *Superseded on 2026-10-30:* star counts **were** re-measured for the twelve projects and the survey is published in §"How the twelve were chose" — the finding is that they do not predict fidelity. Issue counts remain out of scope.

The Corpus: fixture/complex.docx

35 parts, written as hand-checked OOXML with correct CT_PPr / CT_RPr element ordering. It contains, deliberately:

Area	Cases
Tracked changes	w:ins, w:del + w:delText, w16du:dateUtc, w:rPrChange, w:pPrChange, paragraph-mark revision (w:pPr/w:rPr/w:ins), w:moveFrom/w:moveTo with range markers, row-level w:trPr/w:ins and w:trPr/w:del
Fields	complex field (w:fldChar begin/separate/end + w:instrText TOC), w:fldSimple (REF, PAGE), bookmarks
Content controls	5 w:sdt: rich text with w:lock + w:placeholder, date picker, data-bound (w:dataBinding), nested sdt-in-sdt
Markup	w:proofErr ×4, mc:AlternateContent with mc:Choice (DrawingML shape) and mc:Fallback (VML text box), unknown w14:* attributes *and* elements, OMML math (inside a w:p, as Word writes it), w:permStart/w:permEnd, w:altChunk
Tables	tblGrid, gridSpan, vMerge restart+continue, tblHeader, tblLook, nested table, cell shading, textDirection
Layout	3 sections, landscape section with its own header, 2 columns, page borders, docGrid, pgMar
Formatting	w:bidi/w:rtl, w:sym, w:tab, w:br, w:cr, noBreakHyphen, softHyphen, w:vanish, w:noProof, w:vertAlign, eastAsianLayout, 9-level numbering.xml, xml:space="preserve" on 6 whitespace-significant runs
Parts	chart (with embedded workbook), SmartArt diagram (4 parts), OLE object (a real embedded .docx, ProgID="Word.Document.12"), image, glossary/building blocks, customXml, footnotes, endnotes, comments, 2

Area	Cases
	headers, 2 footers, documentProtection, evenAndOddHeaders

Fixture correction worth recording. The first build of the corpus emitted six runs with leading/trailing whitespace but *without* xml:space="preserve" on four of them. OOXML permits a consumer to trim boundary whitespace when that attribute is absent, so eigenpal correctly trimmed them while Folio conservatively kept them — an artefact of the fixture, not of the editors. The corpus was rebuilt so every whitespace-significant run carries the attribute, as Word always writes it, and **all results below are from the corrected corpus.** This is a measurement trap worth naming: a fidelity benchmark that does not mark its whitespace as significant will report the strictest editor as the most lossy one.

Harnesses: the exact code path each project was driven through

Every editor was driven through its **own** public entry point. This is the most important reproducibility asset of this edition.

Editor	Path exercised	Entry point [CODE]
eigenpal	readOoxmlPackage → writeOoxmlPackage (canonical store lane)	packages/core/src/store/package/ooxml-package.ts:418 / :624
Folio	parseDocx → repackDocx (model lane, DOM-free)	packages/core/src/docx/parser.ts:426 / packages/core/src/docx/rezip.ts:1901
GenOffice	parseDocx → Tiptap Editor → pmDocToSavePlan → saveDocx	packages/docx-engine/src/parse.ts:360, apps/docs/src/renderer/editor/convert.ts:139/:2196, packages/docx-engine/src/patch.ts:723
SuperDoc	SuperDocClient.open() → doc.save() (headless SDK + native engine)	@superdoc/sdk 2.25.0-next.14; engine @superdoc/docx-engine 0.27.0-next.15
WordCraft	wordcraft_docx::read → wordcraft_docx::write	crates/docx/src/read.rs:121 / crates/docx/src/write.rs:143
office-open	parseDocument → generateDocument	packages/docx/src/parse.ts:501 / packages/docx/src/generate.ts:71
Docen	parseDOCX → generateDOCX	packages/docx/src/converters/docx.ts:1337 / :1415
canvas-editor	executeImportDocx → executeExportDocx (jsdom + vitest-canvas-mock)	packages/docx/src/docx/importDocx.ts:1807 / packages/docx/src/docx/exportDocx.ts:796
Fidus Writer	DocxConvert.init → DOCXExporter.init (DOM-free)	src/importer/docx/convert.ts:92 / src/exporter/docx/index.ts:34
docs.plus	importDocx → exportDocx (HTML bridge)	.../document-conversion/domain/docxImport.ts:43 / docxExport.ts:30
Plate.js	importDocx → exportDocx (headless, 45 base plugins + Plate's own static presentation kit)	packages/platejs/src/docx/import/lib/importDocx.ts:2427 / export/lib/exportDocx.tsx:646

Four findings about the harnesses themselves:

- **Folio needs preloadFonts: false** to stay headless; otherwise parseDocx reaches for document.fonts and document.createElement("canvas") (packages/core/src/utils/fontLoader.ts:94,99). The repo's own corpus invariants always pass that flag.

- **GenOffice's CLI lane (packages/cli/src/formats/docx.ts:211) cannot be loaded outside a bundler**: it reaches the @genoffice/ui React barrel, whose CSS import and classic-JSX output break a plain Node loader. The harness used the same openDocument/saveDocument logic against the renderer modules directly (convert.ts imports only @genoffice/docx-engine and ../i18n/locale).
- **The SuperDoc headless path is the published SDK, not the repo.** packages/sdk and apps/cli do not exist in this checkout; createDocumentHost/openDocumentRuntime live in the private engine (scripts/engine-native-runtime.mjs:11-19 lists them as runtime exports). The SDK + native binary are published to npm and were used directly.
- **Plate's DOCX path requires a DOM realm, and happy-dom is not enough on its own.** captureDocxDomRealm() (import/lib/importDocx.ts:225-240) throws "DOCX import requires a complete browser DOM realm." unless DOMParser, XMLSerializer, DOMException, HTMLElement and Node.TEXT_NODE are all present. happy-dom supplies them — and the source comments name it explicitly (import/lib/wordFormatting.ts:279, export/lib/reviewPart.ts:120). But happy-dom's GlobalRegistrator **also replaces the global WritableStream and Blob with non-spec-compliant ones**, and @zip.js/zip.js — which reads the package — calls writable.getWriter(), which happy-dom's Writable class does not have. The harness therefore registers happy-dom's DOM and then puts **Node's native WritableStream, ReadableStream, TransformStream and Blob back** (harness/plate/setup_dom.ts). Plate's own headless story for HTML is the same idea with linkedom (packages/platejs/src/html/server/index.ts), but that path is not wired into DOCX.

Measured Results

Round-trip scorecard

Editor	Weighted score	Critical features	Text retention	Package parts kept	Failed checks
SuperDoc V2	**100.0%**	**100.0%** (31/31)	**100.0%**	35 / 35	**0**
Folio	98.5%	96.8% (30/31)	100.0%	35 / 35	1
GenOffice	98.5%	96.8% (30/31)	98.8%	35 / 35	1
eigenpal	98.5%	96.8% (30/31)	97.6%	35 / 35	1
office-open	95.5%	90.3% (28/31)	100.0%	35 / 35	3
Docen	92.0%	83.9% (26/31)	99.4%	35 / 35	6
WordCraft	83.0%	67.7% (21/31)	97.6%	32 / 35	13
canvas-editor + plugin	24.0%	22.6% (7/31)	88.2%	14 / 35	74
Fidus Writer	23.0%	22.6% (7/31)	71.0%	11 / 35	76
docs.plus	21.0%	22.6% (7/31)	91.7%	11 / 35	77
Plate.js	**20.5%**	**19.4% (6/31)**	**91.7%**	**11 / 35**	**77**
LibreOffice (baseline)	*86.0%*	*77.4% (24/31)*	*97.6%*	*26 / 35*	*12*

Weighted score = each of 97 checks scored by criticality (3/2/1). "Package parts kept" counts non-directory entries present in both the input and the output. LibreOffice was converted headlessly (soffice --convert-to docx) and is shown as the calibration floor, not as a candidate.

What each project actually lost

SuperDoc V2 — nothing. 97/97 checks. All 35 parts present, none added. word/document.xml grew 18,276 → 19,513 bytes: a genuine re-save, with document.xml, comments.xml, endnotes.xml, footnotes.xml, header1/2.xml and footer1/2.xml re-serialized and the other 27 parts passed through byte-identically. Paragraph text is identical, including w:ins/w:del text, mc:Fallback text and vMerge continuation text. **The binary payload is intact and still wired up:** all ten payload parts — the chart and its style/colour companions, all four SmartArt parts, the OLE object, the embedded workbook and the image — are present, and the body still references the chart, the diagram and the OLE object. Seven of the 97 checks are exactly that (pkg.chart, pkg.smartart, pkg.ole, pkg.media, ref.chart, ref.smartart, ref.ole), and SuperDoc passes all seven. [MEASURED]

Folio — 1 check. Loses w14:paraId/w14:textId on run properties. Text is 100% intact. The loser's cause is narrow: Folio models the w14 namespace for the *document* but does not round-trip w14 identifiers on w:rPr children. [MEASURED]

GenOffice — 1 check. The visible text stream differs: the text inside the two w:vMerge="continue" cells is dropped. The save plan showed 40 blocks kept as original and **1 regenerated as xml** — the table block, because its block signature did not match. So GenOffice's "verbatim copy of untouched blocks" [RELEASE/ISSUE #2279] holds for 40 of 41 blocks and falls back to regeneration for the one whose signature differs, exactly as convert.ts:2698 describes. Note this matches Word's own merge semantics (Word discards content in vertically-merged continuation cells), so this is defensible behaviour, but it is still a text loss. [MEASURED]

eigenpal — 1 check. mc:AlternateContent is flattened: the mc:Choice branch is kept **and duplicated** where the mc:Fallback was, so "Text box content" appears twice and the fallback's own text is lost. The input's two text-box paragraphs become "Text box contentText box content" and "Text box content". [MEASURED]

office-open — 3 checks. Drops w16du:dateUtc, w14:paraId/w14:textId, and the w16du namespace declaration. Everything else survives, and text is 100% intact. **But it is not idempotent**: feeding its own output back through parseDocument → generateDocument yields word/document.xml that fails to parse with an unbound a: prefix — the chart and SmartArt drawings are emitted as <a:graphic> without rebinding the namespace. Second-pass score collapses to 32.0%. [MEASURED]

Docen — 6 checks. Inherits every office-open loss, and adds its own: w:noBreakHyphen/w:softHyphen dropped, the tracked change inside header2.xml **relocated to header3.xml** (the loss is path-scoped, not content-scoped), and a one-character text difference. Second-pass score 30.5%, same unbound-prefix failure. [MEASURED]

WordCraft — 13 checks. Keeps the chart, all four SmartArt parts, the OLE object, the embedded workbook, the image, the glossary, the customXml store and every header/footer. Loses: w:altChunk (element, part and text), mc:AlternateContent, w:proofErr, w:fldSimple, w16du:dateUtc and its namespace, w14:paraId/textId, **row-level revisions** (w:trPr/w:ins, w:trPr/w:del), and one of three w:ins/w:del pairs. The three lost package parts are word/afchunk.html, word/charts/colors1.xml and word/charts/style1.xml (the chart's style/colour companions). [MEASURED]

Fidus Writer — 76 checks. The decisive finding is at **import**: there is no table handling anywhere in src/importer/docx/*, so both fixture tables (11 rows, 16 cells) are gone before export even begins. The importer also logs 12 "unhandled element" classes: w:proofErr ×4, w:ins, w:del, w:moveFrom/w:moveTo and their range markers, w:bookmarkStart/End, w:fldSimple ×2, w:permStart/End. The exporter is **placeholder-driven**: src/exporter/docx/render.ts fills the template's placeholders — {title}, {@<part-id>} for rich-text and table parts, plus {@bibliography}, {@copyright} and {@licenses} (src/exporter/template/tags.ts:18-101) — so the export cannot carry parts the model does not represent, and the exported styles, numbering and settings are the *template's*, not the corpus's. [MEASURED]

canvas-editor ⋆ plugin — 74 checks. The importer is a bespoke OOXML parser over JSZip (packages/docx/src/docx/importDocx.ts:413, 1,859 lines) and the exporter **rebuilds** the document with the docx npm package (exportDocx.ts:17-47) — a regeneration from the editor's own element list. It also **crashes**: importDocx.ts:1738 drops w:vMerge continuation cells without recording the occupied grid column, so the table layout engine reads colgroup[3] of a 3-column table and throws TypeError: Cannot read properties of undefined (reading 'width') at TableParticle.ts:724. This is a plain-JS bug and reproduces in a browser. [MEASURED]

docs.plus — 77 checks. The pipeline is DOCX → HTML → Tiptap JSON → HTML → DOCX, and the importer is mammoth. The loss is total for revision data, and the code proves it: mammoth/lib/docx/body-reader.js:447 unwraps w:ins and :450 unwraps w:moveTo (insertions become ordinary text), while :729 sets ignoreElements["w:del"] = true and :732 sets ignoreElements["w:moveFrom"] = true (deletions and move-outs are dropped outright), and :727 drops w:sectPr entirely. Counted across the chain (fixture → mammoth HTML → rendered HTML → Tiptap JSON → output): w:ins 3→0→0→0→0, w:del 3→0→0→0→0, w:moveFrom 1→0→0→0→0. **No author, date, or w16du:dateUtc survives anywhere downstream.** [MEASURED]

Plate.js — 77 checks. The pipeline is DOCX → HTML → editor model → static HTML → DOCX, and the losses are the pipeline's shape rather than a bug. **Every revision is flattened** (w:ins, w:del, w:delText, w:moveFrom, w:moveTo and their range markers all fail), **every field is dropped** (w:fldChar, w:instrText, w:fldSimple), **every content control is unwrapped** (w:sdt, w:dataBinding, the date picker, the nested control, w:lock), and mc:AlternateContent collapses to a single branch. All four headers and footers, both notes parts, the comments part, the glossary part, the customXml store, the altChunk part, **the image**, the chart, the four SmartArt parts, the OLE object and the embedded workbook are gone: 24 of 35 parts dropped, 11

regenerated, **none byte-identical**. What survives is the text — **91.7%**, the second-highest in the trial — plus the table grid, gridSpan, vMerge, nested tables, cell shading, bookmarks, w:vertAlign and xml:space="preserve". The three sections become one, so the document renders in **2 pages** instead of 6. [MEASURED]

Plate.js is the only editor in the trial that reports these losses. The import emits **38 typed diagnostics** — 19 unsupported-content, 6 converter-message, 2 resource-omitted, 8 property-dropped, 2 lossy-content and 1 source-unavailable — naming each dropped construct. Under its default lossPolicy: 'reject' those are **18 errors** and the import **fails**: the corpus is refused rather than degraded. No other editor in this trial says anything at all. [MEASURED]

Second pass (idempotence)

Feeding each output back through its own editor is the strictest test: a selective-save editor should reach a fixed point, and its own output should still be valid XML.

Editor	First pass	Second pass (scored against its own first pass)	Byte-identical?	Verdict
SuperDoc	100.0%	**100.0%**	**yes**	Perfect fixed point
Folio	98.5%	98.5%	**yes**	Fixed point
eigenpal	98.5%	100.0%	**yes**	Fixed point
GenOffice	98.5%	100.0%	**yes**	Fixed point
WordCraft	83.0%	84.5%	no	Stable but not byte-identical; text 100%
Plate.js	**20.5%**	**20.0%**	no	Stable, not byte-identical; the second-pass package shrinks from 42,964 to **8,010** bytes as the retained source is replaced by a regenerated one
office-open	95.5%	**32.0%**	no	**Not idempotent — word/document.xml fails to parse: 'unbound prefix', column 13606**
Docen	92.0%	**30.5%**	no	**Not idempotent — 'unbound prefix', column 15858**

Four of the seven editors reach a **byte-identical fixed point** — feeding their output back through their own code path reproduces the same file exactly. WordCraft reaches a stable fixed point in content but not in bytes. office-open and Docen do not merely degrade on the second pass: **the file they emit cannot be parsed as XML at all.** The cause is precise: the chart and SmartArt drawings are serialized as <a:graphic> without rebinding the a prefix, because the namespace is declared on an ancestor element that the regenerated tree does not reproduce.

That is the single most consequential defect found in this trial, because it is invisible in a first-pass diff — the file looks fine, opens in LibreOffice, and only breaks when the tool's own output is fed back in.

Root cause, located. In office-open's *first* pass all four <a:graphic> elements carry a local namespace binding (<a:graphic xmlns:a="http://schemas.openxmlformats.org/drawingml/2006/main">). In its *second* pass the **chart** and **SmartArt** <a:graphic> elements lose that binding and are emitted bare as <a:graphic>, while the **image** one keeps it. The regenerated <w:document> root declares no a prefix, so the bare element is an **unbound prefix** and the file is not well-formed XML. The defect is therefore in the chart/diagram regeneration path, which does not re-emit the local namespace binding that its own first-pass output used. Docen inherits it verbatim, because Docen delegates to office-open.

Word Compatibility of the Corpus

A fidelity corpus that Word itself will not open is not a corpus; it is a curiosity. The first build of complex.docx opened cleanly in LibreOffice and was schema-clean under a naive reading, but **Microsoft Word 16.115 declined it outright**.

Finding out *why* meant treating Word as the oracle and bisecting against it. build/find_word_culprit.py splits word/document.xml into its 41 top-level body blocks, keeps a subset, and asks Word to open the result; build/word_probe.py re-probes one body at a time, and build/isolate_candidates.py / build/move_shape_candidates.py / build/toc_candidates.py / build/dirty_candidates.py narrow the offending markup down to a single attribute. **Twelve defect classes** were behind the refusal.

Five are element-order errors, caught by the ECMA-376 Transitional XSDs and fatal to Word:

Defect	Where	Consequence in Word
w:documentProtection placed after w:defaultTabStop/w:evenAndOddHeaders	word/settings.xml	CT_Settings is a sequence; Word rejects the part
w:tcW emitted after w:gridSpan/w:vMerge	every w:tcPr	CT_TcPr is a sequence; Word rejects the cell
w:cantSplit after w:tblHeader	w:trPr	CT_TrPr is a sequence; Word rejects the row
w:spacing after w:jc inside w:pPrChange	w:pPrChange/w:pPr	CT_PPrBase is a sequence; Word rejects the revision
w:pgNumType after w:cols	the landscape w:sectPr	CT_SectPr is a sequence; Word rejects the section

Three are malformed values and nesting: w16cid:durableId="1 234 567 890" (an ST_LongHexNumber is four bytes, so eight hex digits); dgm:style written as a direct child of dgm:styleDef instead of inside a dgm:styleLbl; and an mc:Fallback that nested a w:r inside the outer w:r instead of holding a w:pict directly.

Four are semantic — markup the schema permits, and Word still refuses:

Defect	Where	Why Word refuses it
the OLE object pointed at 64 bytes of filler	word/embeddings/embedded1.docx	Word embeds a Word document as the raw .docx bytes behind o:OLEObject ProgID="Word.Document.12"; the fixture now embeds a genuine minimal .docx
w:delText inside w:moveFrom	the moved-from run	w:delText marks *deleted* text, and belongs to w:del only. A move source carries ordinary w:t. With w:delText the move reads as a second deletion with no matching insertion, and Word rejects the document
m:oMathPara as a direct child of w:body	the display equation	the transitional schema allows a block-level equation, Word does not: an equation has to sit inside a w:p, the way Word writes it
w:dirty with a true value on a w:fldChar	the TOC field's begin character	see below

The w:dirty result is the one worth keeping. w:dirty is an ordinary ST_OnOff attribute of w:fldChar, and Word itself writes w:dirty="true" on the begin character of an out-of-date table of contents. In this build it is nevertheless fatal to the entire document. The measurement is unambiguous and reproducible:

w:fldChar attribute	Word 16.115 verdict
w:dirty="true"	**refused**
w:dirty="1"	**refused**
w:dirty="false"	opened

w:fldChar attribute	Word 16.115 verdict
w:fldLock="false"	opened
attribute absent	opened

It is not specific to the corpus. The .docx of this report is generated by pandoc, and pandoc writes w:dirty="true" on the TOC field of its own output — **Word refused the first build of this report for exactly this reason.** The identical package with that one attribute set to false opens, and the delivered .docx is the fixed one. A .docx that a generator produces can therefore be unopenable in Word for a reason that has nothing to do with the document's content, and no schema validator will report it.

How the rest was found and fixed. LibreOffice is lenient about element order; the ECMA-376 Transitional XSDs are not. The corpus is now validated with xmllint --schema against the 69 schemas that ship with office-open (ooxml-schemas/transitional/), with exactly two classes of deviation excluded, because both are things every genuine Word file contains and the strict schema does not model:

- the markup-compatibility attributes (mc:Ignorable, mc:AlternateContent), which the transitional XSD does not describe at all; and
- elements and attributes in Microsoft's extension namespaces (w14, w15, w16du, w16cid, w16cex), which is precisely what mc:Ignorable exists to permit — and which are the subject of several test cases in this corpus.

After the fixes, build/validate_ooxml.py reports **no real schema violations** across document.xml, settings.xml, styles.xml, numbering.xml, footnotes.xml, endnotes.xml, comments.xml, both headers, both footers, fontTable.xml, webSettings.xml and glossary/document.xml, and the chart, SmartArt and theme parts validate against their own schemas (dml-chart.xsd, dml-diagram.xsd, dml-main.xsd). **The corpus now opens in Microsoft Word 16.115**, and so does every file this edition delivers.

The lesson generalises past this report. LibreOffice opening a file is not evidence that the file is valid; it is evidence that LibreOffice is forgiving. Any DOCX benchmark whose corpus is only checked in one consumer will quietly measure the corpus's leniency instead of the editors' fidelity. All twelve defects above were invisible to eleven of the twelve editors in this trial — they read the file and produced output — and invisible to LibreOffice. The twelfth, **Plate.js**, refused the malformed corpus outright (decode-failed) rather than reading it; it detects exactly one of the twelve classes, the block-level m:oMathPara. A schema validator caught five of them. The other seven were found only by asking Word, one subset at a time.

Executive Summary: Rankings

1. OOXML fidelity and preservation (measured)

1. **SuperDoc V2.** 100% on the corpus, 31/31 critical, 100% text, no part lost, genuine re-save with byte-identical passthrough of untouched parts.
2. **Folio.** 98.5%, 100% text, no part lost. Loses only w14:paraId/textId.
3. **GenOffice.** 98.5%, all 35 parts kept, one block regenerated.
4. **eigenpal.** 98.5% on features but 97.6% text: mc:AlternateContent flattened to mc:Choice.
5. **office-open.** 95.5%, 100% text, but regeneration drops w16du/w14 metadata and it is not idempotent.
6. **Docen.** 92.0%, inherits and extends office-open's losses.
7. **WordCraft.** 83.0%. Keeps the chart, all four SmartArt parts, the OLE object, the embedded workbook and the image — but it is the only payload keeper that loses the revision model, and it drops two payload parts of its own (word/charts/style1.xml, word/charts/colors1.xml).
8. **canvas-editor ⬧ plugin, Fidus Writer, docs.plus, Plate.js.** Not round-trip pipelines. Plate is the most interesting of the four: its DOCX engine is a well-built, diagnosable, headless import/export pair with a byte-exactness gate, but it is an HTML-shaped pipeline — **zero parts survive byte-identically, 24 of 35 are dropped, and the document renders in 2 pages instead of 6** — so it scores 20.5% while keeping 91.7% of the text. Its diagnostic vocabulary and its lossPolicy: 'reject' default are the best behaviour on malformed input in this trial; see the companion report.

On popularity [MEASURED]. Star counts for all twelve are in §"How the twelve were chosen". **They do not predict fidelity.** The two most-starred projects in the trial, WordCraft (16,740★) and Plate (16,680★), score 83.0% and 20.5%. The winner has 1,117★, and the runner-up has **31★** — fewer stars than the project's own issue count. If you are choosing an editor on popularity, this trial gives no support for that heuristic.

2. Document comparison

Unchanged from the previous report — no comparison code was exercised by this trial, because the trial measured round-trip fidelity, not redlining. Folio's compareDocx and GenOffice's compareDocuments remain the only two implementations found in source (packages/docx-engine/src/compare.ts:108, packages/core/src/compare/compare.ts per PR #694). Note that **neither was tested against the corpus**, and the w:trIns/w:trDel vocabulary used in the previous report is not standard OOXML — the standard row-revision form is w:trPr/w:ins and w:trPr/w:del, which is what the corpus uses.

3. Pagination fidelity

Unchanged. WordCraft's crates/layout remains the deepest page-layout engine; eigenpal's DOM-free layout/ is second. Nothing in this trial measured page geometry — the corpus is a fidelity corpus, not a pagination corpus.

4. Conversion breadth

Unchanged. WordCraft (in-process Rust, widest set), Fidus Writer (academic formats), GenOffice (full suite), office-open/Docen (typed generators).

5. Project health

Unchanged in substance. Two corrections from the previous report stand: GenOffice's ee/ directory is **empty apart from a licence file** — ee/README.md says it "is reserved for future enterprise modules … intentionally empty today", and no DOCX code lives there; the DOCX path is entirely in the Apache-2.0 tree.

Master Comparison Table

Project	Canonical repo / licence	In-memory model	Save strategy	Compare	Pagination	Conversions	**Measured round trip**
SuperDoc V2	superdoc/docx-editor; AGPL-3.0	OOXML-backed document model (V2); V1 ProseMirror archived	V2 engine re-serializes the main story; untouched parts pass through byte-identically	Yes (word-level redlines; apply refuses docs with images/fields/comments, issue #4012)	layout-engine DOM painter	PDF	**100.0%** · 31/31 · text 100%
stella/folio	stella/folio; Apache-2.0	ProseMirror over @stll/docx-core typed OOXML	Selective save; unchanged ZIP entries passed through as original compressed bytes	Yes — strongest redline engine	Paginated	DOCX; CLI, VS Code	**98.5%** · 30/31 · text 100%
GenOffice	genspark-ai/genoffice; Apache-2.0	Tiptap + packages/docx-engine	Verbatim copy of untouched blocks; regeneration on signature mismatch	Yes (compareDocuments, ins/del only)	Paged Tiptap	Broad, via CLI	**98.5%** · 30/31 · text 98.8%
eigenpal	eigenpal/docx-editor; Apache-2.0 core + EigenPal Pro	Canonical typed + generic OOXML tree	Canonical serializer; modelled parts re-emitted normalized from the tree	None found	DOM-free layout/	DOCX→MD (open), DOCX→PDF (Pro)	**98.5%** · 30/31 · text 97.6%
office-open	DemoMacro/office-open; office-open umbrella	Typed descriptors, XSD-shaped	Full regeneration from a parsed model	Not found	None	DOCX/PPTX/XLSX/ODF…	**95.5%** · 28/31 · text 100%

Project	Canonical repo / licence	In-memory model	Save strategy	Compare	Pagination	Conversions	**Measured round trip**
Docen	DemoMacro/docen	Tiptap JSON → office-open	Full regeneration	Not found	None	MD/HTML/ DOCX	**92.0%** · 26/31 · text 99.4%
WordCraft	storytold/wordcraft; licence contested (README MIT/Apache-2.0 vs AGPL-3.0-only per hdatf.com)	Rust wordcraft-doc	Regenerate, with byte-preserving merges for some parts; binary payloads pass through	Not found	Full Rust layout and shaping	Widest in-process set, krilla PDF	**83.0%** · 21/31 · text 97.6%
canvas-editor + plugin	Hufe921/canvas-editor + canvas-editor-plugin; MIT	Own element list rendered to canvas/SVG	Plugin export **rebuilds** the package with the docx npm package	Not found	Own canvas paginator	DOCX via plugin, PDF via canvas	**24.0%** · 7/31 · text 88.2%
Fidus Writer	fiduswriter/fiduswriter + document-ts; AGPL-3.0	ProseMirror schema (Fidus JSON)	Placeholder-driven template regeneration	Not found	None	JATS, LaTeX, EPUB, ODT, DOCX, HTML, BITS	**23.0%** · 7/31 · text 71.0%
docs.plus	docs-plus/docs.plus	Tiptap/ProseMirror + Yjs	DOCX → HTML → JSON → HTML → DOCX (mammoth in, @turbodocx/html-to-docx out)	Not found	None	DOCX/MD import; DOCX/MD/ODT export	**21.0%** · 7/31 · text 91.7%
Plate.js	udecode/plate; NOASSERTION	Slate + a plugin-defined schema; DOCX via HTML	Full regeneration: mammoth in, static HTML → juice → DOCX writer out; retainSource admits a byte-exact export only inside a closed passive vocabulary	Not found	None (the editor is a single flow)	DOCX in/out; MD, HTML, plain text	**20.5%** · 6/31 · text 91.7%

Two rows in that table deserve a footnote rather than a ranking. **Plate.js** is the only project here with a *typed diagnostic vocabulary* and a *byte-exactness gate* — internal/sourceEligibility.ts:453-582 decides whether an unchanged document may be exported byte-for-byte, and lossPolicy defaults to 'reject', so a document it cannot represent losslessly is refused rather than degraded. It scores last because it is an **HTML-shaped** pipeline: every construct Word expresses that HTML cannot carry is lost, and 24 of 35 package parts go with it. **docs.plus** is the same shape for the same reason, but without the diagnostics.

Preserved-vs-Dropped Matrix (measured)

Legend: **M** modelled and editable · **P** preserved · **D** dropped · **?** not applicable. Every cell below is a measurement against fixture/complex.docx, not a reading of release notes.

Item	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	canvas	Fidus	docs.plus	Plate.js
Unknown attributes (w14:paraId, textId)	P	**D**	P	P	**D**	**D**	**D**	D	D	D	**D**
w16du:dateUtc	P	P	P	P	**D**	**D**	**D**	D	D	D	**D**
w16du namespace declaration	P	P	P	P	**D**	**D**	**D**	D	D	D	**D**

Item	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	canvas	Fidus	docs.plus	Plate.js
w16cid:durableId on w:num	P	P	P	P	P	P	P	P	P	P	**D**
xml:space="preserve"	P	P	P	P	P	P	P	P	**D**	**D**	P
w:proofErr	P	P	P	P	P	P	**D**	D	D	D	D
mc:AlternateContent Choice **and** Fallback	P	P	P	**D** (Choice only)	P	P	**D**	D	D	D	D
w:ins / w:del runs	P	P	P	P	P	P	**D** (1 of 3)	D	D	**D**	D
w:rPrChange / w:pPrChange	P	P	P	P	P	P	D	D	D	D	D
w:moveFrom / w:moveTo	P	P	P	P	P	P	D	D	D	D	D
Row-level revisions (w:trPr/w:ins, w:trPr/w:del)	P	P	P	P	P	P	**D**	D	D	D	D
Complex fields (fldChar/instrText)	P	P	P	P	P	P	P	D	D	D	D
Simple fields (w:fldSimple)	P	P	P	P	P	P	**D**	D	D	D	D
Bookmarks	P	P	P	P	P	P	P	D	D	D	P
Content controls (w:sdt)	P	P	P	P	P	P	P	D	D	D	D
w:dataBinding, date picker, nested sdt	P	P	P	P	P	P	P	D	D	D	D
Tables: tblGrid, gridSpan, vMerge	P	P	P	P	P	P	P	**D**	**D**	P	P
Nested table	P	P	P	P	P	P	P	D	**D**	P	P
Section count (3) and landscape	P	P	P	P	P	P	P	D	**D**	**D**	D
Page borders / multi-column / docGrid	P	P	P	P	P	P	P	D	D	D	D
RTL (w:bidi / w:rtl)	P	P	P	P	P	P	P	D	D	D	D
w:sym, w:tab, w:br, hyphens	P	P	P	P	P	**D** (hyphens)	**D** (hyphens)	D	D	D	D
w:vanish, w:noProof, vertAlign	P	P	P	P	P	P	P	D	D	D	D
9-level numbering.xml	P	P	P	P	P	P	P	P	P	P	D
Chart part	P	P	P	P	P	P	**P**	D	D	D	D
SmartArt parts (4)	P	P	P	P	P	P	**P**	D	D	D	D
OLE object + embedded workbook	P	P	P	P	P	P	**P**	D	D	D	D
Image	P	P	P	P	P	P	P	P	P	P	D
Headers/footers (2 + 2)	P	P	P	P	P	P	P	**D** (1 of 4)	D	D	D
Tracked change inside a header	P	P	P	P	P	**D**	D	D	D	D	D
Footnotes / endnotes / comments	P	P	P	P	P	P	P	P/D	D	D	D
documentProtection,	P	P	P	P	P	P	P	D	D	D	D

Item	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	canvas	Fidus	docs.plus	Plate.js
evenAndOddHeaders											
w:altChunk (element + part + text)	P	P	P	P	P	P	**D**	D	D	D	D
Glossary / building blocks	P	P	P	P	P	P	P	D	D	D	D
customXml data store	P	P	P	P	P	P	P	D	D	D	D
docProps/core.xml metadata	P	P	P	P	P	P	P	P	**D**	P	D
Output re-opens in LibreOffice	yes	yes	yes	yes	yes	yes	yes	yes	yes	yes	yes

Per-Repository Findings

1. SuperDoc V2 — superdoc/docx-editor

Pipeline [CODE]. packages/superdoc/src/public/index.ts:4-6 states that the v2 branch exposes a single root package surface and that "V1 editor, converter, zipper, toolbar, and UI compatibility subpaths are not part of superdoc@2". No super-converter, importDocx or loadDocument remains in packages/; the grep for prosemirror in packages/ returns only a bundle-audit script and one collaboration test. The import seam is SuperDoc.ts:3938 replaceFile(source); the export seam is SuperDoc.ts:3986 exportEditorsToDOCX(...) delegating to v2-editor-runtime-adapter.ts:828-831 host.save({ format: 'docx', ... }).

Headless. The V2 model needs no DOM (README.md:84-88: "It reads progressively, renders bounded windows, runs without a browser DOM"); rendering does. The headless lane is the published @superdoc/sdk (SuperDocClient → open({doc}) → document.save({out, force:true})), used exactly as examples/sdk-batch/batch.ts:14-16 does, with the reopen-verification pattern from apps/docs/scripts/execute-migration-snippets.mjs:138-181.

Measured [MEASURED]. 97/97. The only project with no failed check. word/document.xml was genuinely re-serialized (18,194 → 19,409 bytes) and the other 27 parts came through byte-identically. Re-feeding that output through the SDK reproduces the file **byte for byte** — the strongest idempotence result in the trial.

Pros: the only project that survived this corpus unchanged; genuine selective save; headless SDK; company-backed. **Cons:** AGPL-3.0; "Only collaborators can create PRs"; compare/apply refuses documents with inline images, fields or comments (issue #4012).

2. stella/folio

Pipeline [CODE]. parseDocx (packages/core/src/docx/parser.ts:426) → repackDocx (packages/core/src/docx/rezip.ts:1901); the model kernel's serializeDocumentToDocx (packages/docx-core/src/serialize/docx.ts:148) is for *authoring*, not round-tripping. FolioDocxReviewer (packages/core/src/ai-edits/headless.ts:1110, fromBuffer at :1560, toBuffer at :3474) is the object-model lane. parseDocx must be called with { preloadFonts: false } to stay DOM-free (parser.ts:834-840).

The repo's own harness was reused. scripts/lib/corpus-invariants/save-idempotence.ts:24-27,137-141 does exactly the parse → repack → compare cycle this trial used, and scripts/lib/corpus-invariants/editor-round-trip.ts:19-22,43-104 does the full editor-model round trip. scripts/lib/corpus-invariants/reserialize.ts:6-12 explains why a first save is never a no-op: every repack runs normalizeRevisionIdsInXmlParts and normalizeParaIdRangeInXmlParts over every word/*.xml part, so "comparing the input package against its first save would report that normalisation rather than any defect". Folio's docs/docx-platform.md:137 claims "Unknown elements, attributes, relationships, and package parts are preserved by default".

Measured [MEASURED]. 98.5%, 100% text, no part lost. Sole loss: w14:paraId/w14:textId on run properties.

Pros: strongest open redline engine; release notes that read like fidelity specifications; the only project whose own corpus-invariant harness matches this trial's shape. **Cons:** tiny community (30 stars); very fast-moving 0.x API; effectively a single maintainer.

3. genspark-ai/genoffice

Pipeline [CODE]. parseDocx (packages/docx-engine/src/parse.ts:360) → blocksToPmDoc (apps/docs/src/renderer/editor/convert.ts:139) → Tiptap Editor → pmDocToSavePlan (convert.ts:2196) → saveDocx (packages/docx-engine/src/patch.ts:723). The signature functions are signatureOfBlock at convert.ts:4070 and signatureOfGenerated at convert.ts:4091, used at :2698 — **note this corrects the previous report, which cited convert.ts:2155-2189 and :2381-2417 for these; those line ranges are Zotero-field fixup and image-patch logic respectively.**

No docx→docx CLI route exists. packages/cli/src/commands/convert.ts:31,47 map docx: ['md'] and docx: ['pdf','html'], and :91-98 raises "cannot convert .docx to .docx". The nearest is genoffice docs apply in.docx --ops ops.json --out out.docx (packages/cli/src/commands/docs.ts:314), which rejects an empty ops list at :290-297.

The repo's own no-op tests. packages/docx-engine/tests/roundtrip.test.ts:19-26 asserts "no edits → output is the exact original file (byte identical)", and apps/docs/tests/inline-rule.test.ts:318-327 asserts the same through the Tiptap editor.

Measured [MEASURED]. 98.5%, all 35 parts kept, 40 of 41 blocks kept verbatim, **1 block regenerated** — the table, whose content in w:vMerge="continue" cells was lost.

Known-loss statements in code (unchanged from the previous report, now with a second-pass corroboration): parse.ts:2583-2584 ("inline drawings in textbox paragraphs … are dropped — and lost on first edit"), parse.ts:4403-4405 (a run carrying several drawings must split, "or every picture past the first is dropped — and permanently lost"), generate.ts:732 ("rewriting the outer text of a cell containing a nested table is not supported yet"), zip-splice.ts:90 ("zip64 archives are not supported"), types.ts:1872-1875 (WordArt is "never written to OOXML").

Pros: a whole suite; a usable redline CLI; outside PRs merge fast; strongest non-fidelity feature breadth. **Cons:** the regeneration fallback is lossy; ee/ signals a planned enterprise split (currently empty).

4. eigenpal/docx-editor

Pipeline [CODE]. readOoxmlPackage (packages/core/src/store/package/ooxml-package.ts:418) / writeOoxmlPackage (:624); part level readOoxmlPart (ooxml-tree.ts:2006) / serializeOoxmlPart (ooxml-serialize.ts:411). The public surface is @docx-editor.dev/core/store (packages/core/package.json exports "./store"). The headless object-model lane is createServer (packages/editor-api/src/runtime/server.ts:159) with save() at runtime.ts:547.

xml:space is modelled and normalized, not preserved verbatim. OoxmlXmlSpaceAttribute is declared at ooxml-tree.ts:96-102 ("xml:space, which decides whether a run's leading and trailing whitespace survives"), and normalizedWmlTextAttributes (ooxml-shared.ts:601-620) *re-derives* the attribute from the text: it drops a redundant or stale xml:space and injects xml:space="preserve" when wmlTextNeedsXmlSpacePreserve (ooxml-shared.ts:579-587) says the text has boundary whitespace or a tab. The XML reader itself is careful: xml-reader.ts:247-251 configures fast-xml-parser with trimValues: false ("preserve significant whitespace"). **This design is correct and is exactly why the first corpus build produced a false finding** — see § *Fixture correction.*

mc:AlternateContent is processed, not preserved. The tree models MC (MC_QNAME_LIST_ATTRIBUTES = ProcessContent, PreserveElements, PreserveAttributes, ooxml-shared.ts) and ooxml-package.ts:616-619 states the design intent: "The modeled parts are re-emitted NORMALIZED from the tree rather than patched as text." Measured, that normalization resolves the MC choice and discards the fallback.

Measured [MEASURED]. 98.5% on features, **97.6% text** — the only top-four project that loses visible text. mc:AlternateContent becomes "Text box contentText box content" + "Text box content".

Pros: the most principled fidelity *model*; Vue and React; agent tooling. **Cons:** measured third, not first; review features are paid; the MC fallback loss is silent.

5. DemoMacro/office-open

Pipeline [CODE]. parseDocument (packages/docx/src/parse.ts:501) / generateDocument (packages/docx/src/generate.ts:71); umbrella at packages/office-open/src/document.ts:88,112. The repo's own round-trip test is packages/docx/src/generate.spec.ts:515. DOM-free.

Measured [MEASURED]. 95.5%, 100% text, all 35 parts present — but w16du:dateUtc, w14:paraId/textId and the w16du namespace are dropped, and **the pipeline is not idempotent**: its own output re-parses with an unbound a: prefix (second pass 32.0%). The exact fault is that the chart and SmartArt <a:graphic> elements lose the local xmlns:a binding on the second pass while the image one keeps it, and the regenerated root declares no a prefix.

DemoMacro/office-open is upstream; akademi-ink/office-open is a fork at v0.14.5 that no longer tracks upstream and ships only core/docx/pptx/xml. diff -rq over packages/docx/src reports 195 differing or extra files. The previous report cited both URLs interchangeably.

6. DemoMacro/docen

Pipeline [CODE]. parseDOCX (packages/docx/src/converters/docx.ts:1337) / generateDOCX (:1415), delegating to @office-open/docx 0.15.0.

Measured [MEASURED]. 92.0%, 99.4% text, no part lost but four new header/footer parts. Inherits office-open's metadata losses and adds w:noBreakHyphen/w:softHyphen loss and relocation of the tracked change inside header2.xml to header3.xml. Same non-idempotence (second pass 30.5%).

7. storytold/wordcraft

Pipeline [CODE]. wordcraft_docx::read (crates/docx/src/read.rs:121) / write (crates/docx/src/write.rs:143); the engine's file layer is open_bytes / save_bytes (crates/engine/src/io.rs:144,374). The CLI is wordcraft-cli convert <in> <out>. Building only wordcraft-docx needs no git dependency, no GPU, no fonts, no layout — the trial did exactly that, and the crate built in 23 s.

Measured [MEASURED]. 83.0%, 97.6% text. It keeps the **chart, all four SmartArt parts, the OLE object, the embedded workbook and the image** — the only one of the seven projects that keep those payloads that also loses the revision model: w:proofErr, w:fldSimple, w:altChunk, mc:AlternateContent, w16du:dateUtc, w14 identifiers and **row-level revisions** all disappear. Three package parts are lost: word/afchunk.html, word/charts/colors1.xml, word/charts/style1.xml.

Self-reported fidelity [DOCS]. docs/file-format-parity.md:49 still says **~65%** for .docx R/W, "synthetic fixtures only", "Untested on a real-world corpus"; :10-13 discounts the weighted table to "~60% because DOCX fidelity has never been checked on real-world files". docs/target-app-parity.md:47-49 names opening real-world .docx files and comparing pages as the **#1 open gap**. This trial is one such check, and 83% against a deliberately hostile document is consistent with the ~65% self-assessment for ordinary files.

8. Hufe921/canvas-editor ✦ canvas-editor-plugin

Pipeline [CODE]. Import is a bespoke OOXML parser over JSZip (packages/docx/src/docx/importDocx.ts:413, 1,859 lines) — **not mammoth.js**, correcting the previous report's uncertainty. Export **rebuilds** the package with the docx npm package (exportDocx.ts:17-47). Registration is packages/docx/src/docx/index.ts:5-10.

DOM requirements [CODE]. DOMParser (XML mode), atob, URL.createObjectURL/revokeObjectURL, Blob, Image, document.createElement('canvas').getContext('2d') for measureText (packages/docx/src/docx/utils.ts:40,70), and document.body for saveAs (:10-20) — avoidable with download: false. The repo's own headless recipe is canvas-editor/tests/factories/editor.ts:50-77 plus tests/setup.ts; the trial used an equivalent vitest project with vitest-canvas-mock.

Measured [MEASURED]. 24.0%, 88.2% text, 21 of 35 parts lost. The importer is the degrading side: 12 input paragraphs are already missing from the editor's in-memory model, while **0** model paragraphs are missing from the output — the exporter loses no text, it only rewrites structure (list numbering flattened to literal "1."/"2.", w:sdt flattened to plain text, footnotes/comments emitted as empty stubs, 1 of 3 sections).

A real crash. importDocx.ts:1738 drops w:vMerge continuation cells without recording the occupied grid column; the layout engine then re-derives colIndex off by one and reads colgroup[3] of a 3-column table: TypeError: Cannot read properties of undefined (reading 'width') at canvas-editor/src/editor/core/draw/particle/table/TableParticle.ts:724, via LayoutEngine.ts:296 ← Draw.render (Draw.ts:2664) ← Draw.setValue (:1688). Plain JS, so it reproduces in a browser.

9. Fidus Writer

Pipeline [CODE]. DocxConvert (src/importer/docx/convert.ts:92) → DOCXExporter (src/exporter/docx/index.ts:34, init() at :72, download(blob) at :168), with a template from generateDocxTemplate (src/exporter/template/docx.ts:250). DOM-free unless the document has citations (src/exporter/docx/citations.ts:112-130 uses prosemirror-model DOMSerializer/DOMParser). The CLI equivalent is fidusconvert in.docx out.docx (src/commands/convert.ts:376-384 import, :283-290 export).

Measured [MEASURED]. 23.0%, 71.0% text, 24 of 35 parts lost. The decisive defect is at **import**: src/importer/docx/* has **no table handling**, so both fixture tables (11 rows, 16 cells) are gone before the exporter runs. The importer also reports 12 unhandled element classes and silently loses w:ins/w:del/w:moveFrom/w:moveTo/w:rPrChange/w:pPrChange/w:bookmarkStart/w:fldSimple/w:permStart/w:proofErr. The exporter is placeholder-driven (src/exporter/docx/render.ts), so it can only fill the template's slots — {title}, {@<part-id>}, {@bibliography}, {@copyright}, {@licenses} (src/exporter/template/tags.ts:18-101) — and the exported styles, numbering and settings are the template's own.

A methodological note worth publishing. Using the *input document itself* as the export template scores 99.5% — because the input contains no {@<part-id>} placeholder, the Fidus model is never written and word/document.xml comes through unchanged. That is an artefact of a template-driven exporter, not a round trip, and it is the kind of number that could be published in error. The reported 23.0% is the genuine model-based round trip.

10. docs-plus/docs.plus

Pipeline [CODE]. importDocx (apps/hocuspocus.server/src/modules/document-conversion/domain/docxImport.ts:43) → exportDocx (docxExport.ts:30), bridged by renderDocumentHtml / parseDocumentHtml (documentHtml.ts:38,77) and getRenderSchema/getParseSchema (conversionSchema.ts:33,35). No DB, Yjs, Redis or auth is needed for these domain functions — Prisma and the access gate live only in http/controller.ts. The pipeline is **DOCX → HTML → Tiptap JSON → HTML → DOCX**: mammoth in, @turbodocx/html-to-docx out.

Measured [MEASURED]. 21.0%, 91.7% text, 24 of 35 parts lost. **Tracked changes are destroyed at the mammoth import stage**, with the code to prove it: mammoth/lib/docx/body-reader.js:447 unwraps w:ins (insertions become ordinary text), :450 unwraps w:moveTo, :729 sets ignoreElements["w:del"] = true and :732 sets ignoreElements["w:moveFrom"] = true (deletions and move-outs are dropped outright), :727 drops w:sectPr entirely, :728 drops w:proofErr, and :500 only tests trPr/w:del, so an inserted row silently becomes a normal row. Counts across the chain: w:ins 3→0→0→0→0; w:del 3→0→0→0→0; w:moveFrom 1→0→0→0→0.

Pros: collaboration and an MCP connector. **Cons:** an HTML bridge, not a DOCX tool; every revision is flattened at import with no warning to the user.

11. udecode/plate — Plate.js (added 2026-10-30)

Plate is the most popular rich-text framework in the JavaScript ecosystem, and it is **not** a DOCX editor in the shape the other eleven here are. It is a Slate-based editor framework with a DOCX engine bolted on — and that engine is the most carefully built one in this trial.

Pipeline [CODE]. A headless import/export pair, both in packages/platejs/src/docx/:

- importDocx(arrayBuffer, { plugins, lossPolicy, retainSource }) – import/lib/importDocx.ts:2427. It throws TypeError('DOCX import requires a plugins array.') without plugins (:2437), and lossPolicy defaults to **'reject'** (:2455).
- exportDocx(editor, { projection, source, lossPolicy, presentation, stylesheet }) – export/lib/exportDocx.tsx:646. projection is mandatory (:650); the result is a Blob (:119-128).

The import path is **mammoth → HTML → the editor model**: mammoth is a top-level import (importDocx.ts:4), and the HTML is then decoded through the plugin set's own formats.html mappings. The export path is the mirror image: **editor model → static HTML → juice → the DOCX writer** (export/lib/internal/html-to-docx.ts). So Plate's DOCX fidelity is bounded by **how much of Word's document model HTML can carry**, and by how completely the plugin set covers the constructs in the file.

The one lever that matters most is presentation. export/lib/internal/docxStaticComponents.tsx:200-211 maps ten plugins to DOCX-native drawings (callout → a two-cell table; columns → a layout table; equations → KaTeX → MathML → OMML; TOC → hyperlink paragraphs). When presentation is omitted, the export falls back to each plugin's own React component — and the **Base** (non-React) plugins carry none, so tables, images and lists silently degrade to plain divs (static/internal/staticPresentation.ts:103). This harness therefore uses Plate's own static kit from apps/www/src/registry/components/editor/*-static.tsx (32 files) together with DOCX_EXPORT_STYLES, which is what the official demo uses.

Measured [MEASURED]. 20.5% weighted, 19.4% critical (6/31), **91.7% text retention**, 11 of 35 parts kept, 77 failed checks. **Zero parts are byte-identical** and 24 of 35 are dropped: every header, footer, footnote, endnote, comment, chart, SmartArt part, glossary part, customXml part, the altChunk part, the image and both embeddings. What remains is a regenerated 11-part package — [Content_Types].xml, _rels/.rels, docProps/core.xml, word/document.xml, word/_rels/document.xml.rels, word/styles.xml, word/numbering.xml, word/settings.xml, word/webSettings.xml, word/fontTable.xml and word/theme/theme1.xml — all changed, none added. The output renders in **2 pages** instead of 6.

But Plate is the only editor here that says what it did. The import produces **38 typed diagnostics**, and they are precise: 19 unsupported-content (field-instruction content replaced, field content replaced, five structured-document-tags unwrapped, embedded-object content replaced, text-box content unwrapped, alt-chunk content dropped, header/footer/footnote/endnote content not mapped into the main document, four move-range markers flattened), 6 converter-message (w:cr, w:fldSimple, o:OLEObject, w:permStart, w:permEnd, w:altChunk unrecognised), 2 resource-omitted (embedded image omitted), 8 property-dropped (background color ×2, font size ×4, font family ×2, caps, small caps, character spacing, alignment ×2, superscript) and 2 lossy-content (tracked revisions flattened; comment 0 has no complete imported range). Under Plate's own default lossPolicy: 'reject' those same diagnostics are **18 errors** and the import **fails** — the corpus is refused rather than silently degraded. The harness runs with 'allow' so that an output exists to measure.

The source-unavailable diagnostic is the interesting one. Plate's retainSource: true keeps the admitted package so an unchanged document can be exported byte-exactly and an edited one source-aware. Admission is governed by internal/sourceEligibility.ts:453-582, which checks that every part, content type, relationship and inspected markup sits in a **closed passive vocabulary**: exactly one officeDocument relationship pointing at word/document.xml (:531-542), no external relationships except absolute hyperlinks (:500-514), raster parts matching their declared image type's magic bytes (:543-547), every part reachable through the relationship graph (:560-567), and only PAGE recognised as a field instruction (:128). The corpus fails that last-but-one test — word/_rels/document.xml.rels declares a **chart** relationship, which is not in the passive vocabulary — so Plate diagnosed:

> source-unavailable: The DOCX source cannot be retained because word/_rels/document.xml.rels declares an unrecognized relationship (chart); export will generate the document from editor content.

Pros: a typed diagnostic vocabulary; a default loss policy that refuses rather than degrades; a passive-vocabulary gate that makes byte-exact export a provable property rather than a hope. **Cons:** it is an HTML-shaped pipeline, so everything Word expresses that HTML cannot carry is lost — which is most of the package; and its DOCX engine is not designed for round-tripping, so the score is low by construction.

Second pass [MEASURED]. 20.0%, 0 errors, and the package shrinks from 42,964 to **8,010** bytes — Plate's own output re-imports cleanly, but the regenerated document no longer carries the parts the first pass had written.

Changes from the Previous Report

Corrected by measurement

- **The fidelity ranking was wrong.** eigenpal was ranked first; measured, it is third behind SuperDoc and Folio, and loses visible text that Folio keeps.
- **GenOffice's DOCX entry points were mis-cited.** signatureOfBlock is convert.ts:4070 and signatureOfGenerated is :4091, not :2155-2189 and :2381-2417.
- **canvas-editor's DOCX importer is not mammoth.js** — it is a bespoke 1,859-line OOXML parser over JSZip.

- **The w:trIns/w:trDel vocabulary is not standard OOXML.** Row revisions are w:trPr/w:ins and w:trPr/w:del. The corpus uses the standard form.
- **GenOffice's ee/ directory contains no code** — only LICENSE and a README.md stating it is intentionally empty.
- **DemoMacro/office-open is upstream of akademi-ink/office-open**, which is a stale fork at v0.14.5.

New, established by this trial

- SuperDoc V2 is the only project with zero failed checks.
- office-open and Docen are **not idempotent** — their own output is malformed XML on re-import.
- canvas-editor's DOCX import **crashes on w:vMerge**.
- Fidus Writer's DOCX importer **has no table support**.
- docs.plus **destroys all tracked changes at import**, via mammoth's ignoreElements.
- WordCraft keeps binary payloads but **loses row-level revisions**.
- **Microsoft Word 16.115 opens the corpus.** Getting it there meant fixing twelve defect classes, seven of which no schema validator reports — among them w:delText inside w:moveFrom, a block-level m:oMathPara, and w:dirty="true" on a w:fldChar. The last one also broke the first build of this report's own .docx.

Still unverified

- Pagination fidelity, comparison/redline quality and conversion breadth were not exercised; those sections carry the previous report's evidence unchanged.
- WordCraft's licence remains contested (README "MIT or Apache-2.0" vs hdatf.com "AGPL-3.0-only").
- The Docen licence was not verified.

Recommendations

- **If you need a document to survive an open-source editor unchanged: SuperDoc V2.** It was the only project to pass the corpus with no losses, and it re-saved the document rather than copying it. Budget for AGPL-3.0 and check the compare/apply limits before committing.
- **If you need a redline engine as well: Folio.** Equal feature score, better text fidelity, and a much more rigorous compare implementation than anything else here. Re-run this corpus on your own documents before you commit, because w14 identifiers are the one thing it dropped.
- **Avoid GenOffice's regeneration path on heavily revised files** until issue #524 is closed — it is the one block in this corpus it could not copy verbatim. For everything else it is a capable suite.
- **Treat office-open and Docen as generators, not round-trippers.** They rebuild the package; if your workflow re-imports their output, note that the result may not even parse. WordCraft is a better choice for payloads (charts, SmartArt, OLE) if you can live without the revision model.
- **Do not use Fidus Writer, canvas-editor ✦ plugin, docs.plus or Plate.js as fidelity tools.** Fidus Writer is an academic-publishing pipeline, canvas-editor is a canvas editor with a DOCX exporter, docs.plus is an HTML bridge that silently accepts every revision, and Plate.js is an HTML-shaped editor framework whose DOCX engine, however well built, does not carry the constructs Word uses.
- **But do use Plate.js's DOCX engine's *diagnostics* as a model.** It is the only one of the twelve that reports what it dropped, and its default lossPolicy: 'reject' refuses a document it cannot represent losslessly instead of writing a degraded one. If you are building a DOCX pipeline, that behaviour — a typed diagnostic per lost construct, and a refusal by default — is worth copying even if you do not adopt the engine. Its sourceEligibility.ts gate is also the only implementation of *provable* byte-exact export found in this trial.
- **Build your own corpus.** This one document found eight distinct defect classes that no release note mentions. Your documents will find different ones.
- **Do not choose on stars.** The two most-starred projects here score 83.0% and 20.5%; the winner has 1,117★ and the runner-up 31★. Stars track how visible a project is, not whether it can open a Word file.

Caveats

- One adversarial document, not a corpus survey. The scores rank *this* input, not general quality.
- No edits were performed; the trial measures round-trip fidelity only.
- The corpus itself was opened in Microsoft Word 16.115, and every defect that Word refuses was found by bisecting the document against Word. The editors' **outputs** were judged by the scorer and re-opened with LibreOffice, not by Word.

- The comparison and pagination sections were not re-measured and retain the previous report's evidence.
- Scores depend on this corpus's weighting; the raw pass/fail lists are in out/<editor>/score.json so the weighting can be changed.

Reproducing This

```
build/make_fixture.py                # writes fixture/complex.docx (35 parts)
build/validate_ooxml.py fixture/complex.docx   # ECMA-376 Transitional XSD check
build/run_all.sh                     # runs every editor's own import->export path
build/check_roundtrip.py IN OUT --json # 97-check scorer
build/score_all.py                   # aggregate table
build/payload_audit.py               # per-editor chart/SmartArt/OLE/image part + reference audit
build/fix_dirty.py REPORT.docx       # w:dirty="true" -> "false" (Word refuses the former)
build/verify_deliverables.py         # opens the deliverables in Word and reports what Word sees
```

Per-editor harnesses live in harness/ (eigenpal.ts, folio.ts, genoffice.cts, superdoc.mjs, officeopen/, docen/, fidus/, canvas/, wordcraft/, docsplus/, plate/). Outputs and per-check JSON are in out/<editor>/. The **sixteen** upstream checkouts are under repos/, each at the commit pinned in the table above; none of them was modified.

The Word-side tooling lives in build/ as well: find_word_culprit.py bisects word/document.xml against Word, word_probe.py probes one body at a time, wxml.py splits a body into gap-free blocks, and isolate_candidates.py, move_shape_candidates.py, toc_candidates.py and dirty_candidates.py are the isolation experiments behind the semantic-defect table.

Source citations for the code-level claims in this edition are the upstream repositories at those commits, at the file paths and line numbers given inline. The previous edition's numbered source list (68 links) remains applicable to its unchanged sections; this edition adds the local harness and the pinned-commit table as its own evidence base.